# Automated Generation and Verification of SELinux Policies to Enhance Software Security

Faheem Khan Rahman

Supervised by

Dr. Ian Batten

September 2025

# DECLARATION

The code involved in this project, shown in the supplied screenshots and in the GitLab repository is a combination of my own code and code generated or debugged using ChatGPT 4.0 and, upon its release on the 7th August 2025, ChatGPT 5.0.

During debugging, example prompts which were entered into ChatGPT 4.0 and ChatGPT 5.0 were: *"My policy is not compiling, what seems to be this error here?"*, supplied with the error that I was given, to which it responded with a method to fix the program and *"My script is not working properly, why is this?"*, supplemented with the current state of the script code.

ChatGPT 4.0 and ChatGPT 5.0 also assisted in the generation of tables, the flow graph, for the references and for ensuring the overall clarity of this report. Example prompts used here included: *"Can you structure these results into a table?"* and *"Is the wording of this paragraph clear?"*. All generated outputs were reviewed before being included in this document/project. All other written work remains my own, unless cited[1].

The use of generative AI was instrumental for the development in my project, as it was able to fix any gaps I had when programming, such as from programming the script as part of the project to the LaTeX code for this document when structuring tables. Regarding the programming, the code for the initial policy was my own, but for other programming instances, such as the finalised policy and the automated script was completed with some assistance with ChatGPT 4.0 and ChatGPT 5.0, especially for deubgging.

[1]The prompts listed here are non-verbatim but of a very similar nature; in reality these prompts were more brief, usually contained spelling errors and, at times, adapted to the frustration of the author.

# ACKNOWLEDGEMENTS

I would like to thank my supervisor, Dr. Ian Batten, for his supervision of this project, Dr. Mihai Ordean for his feedback following my project demonstration and the many music artists whose music dulled the occasional stress from the completion of this project[2].

[2]Including but not limited to The Police, Phil Collins, Bruce Springsteen and, go on, Nik Kershaw.

# ABSTRACT

This project explores a method for reducing the complexity of manually writing SELinux security policies by automating their generation using trace-based system behaviour. A custom binary was developed to observe how an application interacts with the system during its runtime, extracting the relevant system calls and file paths and then using this information to generate a basic SELinux type enforcement policy module.

An initial SELinux policy for the nano was generated and was refined through the use of audit logs which were collected while the SELinux was set to permissive mode, meaning all actions would be allowed. Allow rules were created using audit2allow and were then applied to the policy. The verification of the policy came in the form of static analysis via Ghidra which was used to identify additional system calls and this was followed by dynamic analysis which was used for the testing whether the enforced rules were applied via command testing and automated scripting.

The results from these testing show that while dynamic analysis provided practical results which are tailored to observed behaviours, static analysis reveals hidden or infrequently used functions which help prevent the appearance of false positives. The use of both static and dynamic analysis highlighted the trade-offs between ease of use and the enforcement of least privilege with the automation being highly dependant on the generated audit logs.

# Contents

# Chapter 1

# Introduction

Security Enhanced Linux (SELinux) is a powerful access control mechanism integrated into the Linux kernel. SELinux enforces access control which restrict how the processes interact with the files and other resources, offering security. However, despite its security, SELinux can be difficult to configure and maintain. Its complex policy language and the steep learning curve associated with writing correct and complex security policies often discourage system administrators from adopting this practise.[1] [2]

One of the factors of effective SELinux implementation is the creation of custom policies for the use by third-parties. While SELinux offers fine-grained control, users have to manually every permission that a user needs. These permissions are defined in type enforcement (.te) files, which must be compiled and installed using SELinux development tools.

It can be a long and complicated process to manually specify every permission that for users. To make this process easier, tools exist which can help make this easier, such as seccomp. However, these tools are limited as they rely on pre-existing audit logs as well as providing limited insight on the programs. Current SELinux analysis tools often add more layers than needed, such as the inclusion of mathematical abstraction, before producing any results, which further complicate the process of generating policies. [3]

This project aims to address these challenges by automating the generation and verification of SELinux policies. This project's approach focuses on observing the system behaviour during controlled executions, extracting invoked system calls and accessed file paths, performing analyses on these policies, thus ensuring a complete set of policy rules. Specifically, this project focuses on the following areas:

- Developing a policy through the use of audits.
- Refining this policy and then performing verification via static and dynamic analyses.
- Evaluating whether the refined policy preserves functionality while denying potentially malicious behaviour.

In doing this, this project aims to demonstrate that automated policy generation is feasible and that the combination of static and dynamic analysis can help improve the robustness of the SELinux policy.

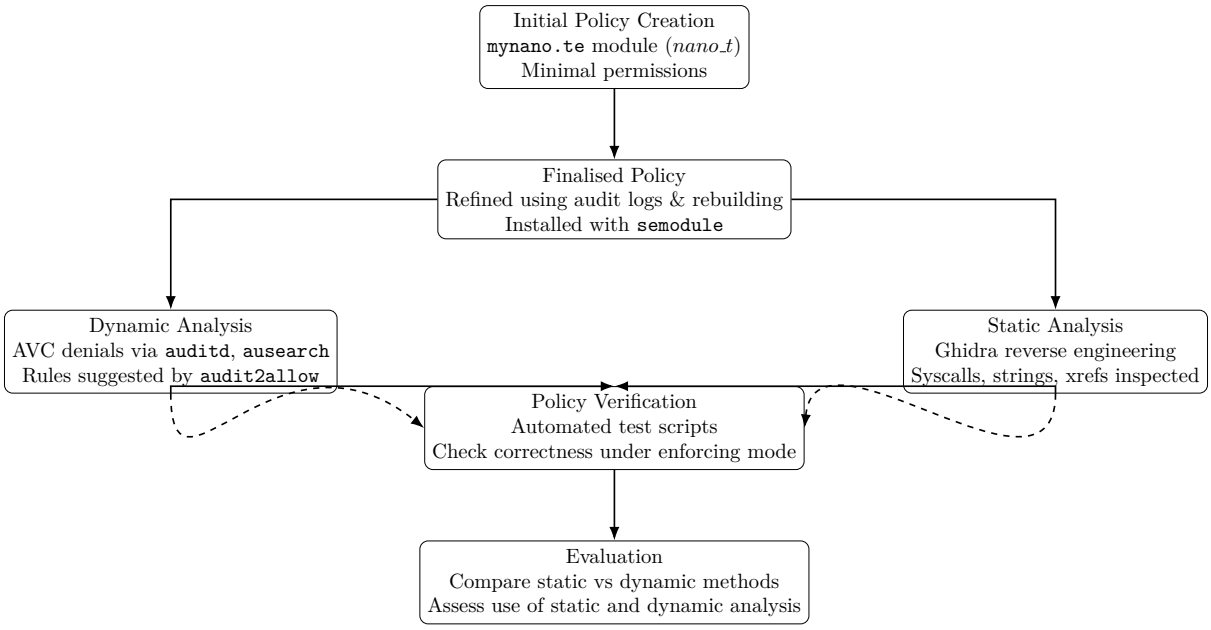


Figure 1.1: Flow graph of policy generation, analysis, and evaluation process

# Chapter 2

# Background

## 2.1 SELinux

SELinux is a security module integrated into the Linux kernel, providing mechanisms for Mandatory Access Control (MAC). Originally developed by the National Security Agency (NSA), SELinux enforces strict access control policies that determine interactions between the subjects (processes) and objects (files and sockets). Unlike other access control mechanisms, such as Discretionary Access Control (DAC), SELinux policies define permissible actions through security contexts assigned to every process and object, reducing risk of privilege escalation in the event of a compromise. SELinux is widely adopted in environments where isolation between processes and precise control of access to resources is critical, supporting customisable policies that can enforce highly specific requirements that are tailored between systems.[4]

However, tailoring requirements to systems does mean that policies can be quite complicated to write and require advanced knowledge and need to be refined regularly to ensure security and functionality. Existing documentation is often outdated and may assume knowledge of advanced terminology, something that not every user possesses. As a consequence of this, the creation of policies may contains bugs or missed with the developers having to use audit logs to patch denials. Automated generation of SELinux policies has been proposed, however the perceived difficulty in doing this render the automation process infeasible.

The challenge has sparked research into policy automation, such as those from [5] and [6], each proposing mechanisms to learn policies of system activity or to extract them from code, however, these approaches are often restricted heavily onto specific tools and those that are unable to generalise across real world applications. Because of this, systems often run SELinux on permissive mode, which allow actions to be performed, which would be denied when SELinux runs on enforcing mode.

## 2.2 Sandboxing

Sandboxing is a security practise designed to limit the actions of a process, in order to reduce the impact of a security vulnerability. Sandboxed processes are restricted to a tightly bound environment where access to tools are monitored. Restricting process to bound environments have been applied in several applications such as container environments like Docker, Linux

kernel mechanisms such as seccomp as well as applications built using Java and JavaScript. Attempts at sandboxing have come with their challenges. Modern sandboxing tools such as seccomp from Linux, which filters system calls, and AppArmor, which feature profile-based restrictions, offer only partial solutions with Alhindi and Hallet (2025) documenting usability and documentation issue with their use of seccomp [7]. Both seccomp and AppArmor are much easier to implement than SELinux, hence their popularity, but both applications lack the specific security that SELinux offers. seccomp allows system calls to be filtered, which can reduce the attack surface of the application and AppArmor offers path bases control access and their policies can be introduced using simple tracing tools while SELinux policies may use multiple abstractions (types, roles and the users). These methods are easier to implement, as opposed to other measures like automating these but these lack the richness which the SELinux policy framework provides.

Other modules, such as the Java Virtual Machine (JVM), have promised strong security isolation, but ultimately they weren't scalable in general purpose applications for reasons such as their complexity and problems with their sustainability [8]. These failures illustrate a broad truth, that scalability is hard to perfect, especially when its complex security makes sandboxing complicated to use. SELinux can be seen as a general-purpose solution to sandboxing, though it is one that does require significant amounts of manual intervention, such as manually reviewing any generated audit logs. SELinux can be considered a general-purpose solution to sandboxing, albeit one that requires a lot of manual intervention and this is something that can be eased with automation.

## 2.3 Automation

The automation part of this project is one thing that is not only beneficial but one that is necessary to enforce role based access control. Manual policy generation is a method that is time-consuming and error-prone and can lead to the production of rules that can be both too strict, which can break functionality, as this could block common actions, or rules that are too loose, which can weaken security. Several tools such as setools to ease this limitation but their outputs can be too vague or overly permissive. Automation introduces a trade-off between usability and security. Tools such as audit2allow can ease refinement but may be able to create false positives. Machine learning approaches such as Jain et al (2025) [9] can detect errors with a high accuracy (95% from their research) but they require curated datasets and introduce interpretability challenges.

## 2.4 Security Contexts and Type Enforcement

SELinux programs feature a labelling system that is built around security contexts. Every process, file, sockets and other kernel objects are assigned the user, role and type components.

### 2.4.1 User

The user field defines the identity which the process or object runs under, from the perspective of the SELinux policy. The user field is mainly used for Role-Based Access Control (RBAC) purposes. A user can be restructured to several roles or can be allowed to have all roles. These are restrictions that limit which actions and permissions that a particular user has.

### 2.4.2 Role

The role field is used to define the set of types or domains, a subject can enter or transition into. Roles are usually relevant for processes. Roles help implement the RBAC by grouping together different types and domains and controlling which domains can be transitioned by the SELinux users. For example, the system_r role is given to many system resources. A staff_u SELinux user may be allowed to have assume the staff_r role, but they cannot assume the sysadm_r rule, unless the have elevated permissions [2].

### 2.4.3 Type

Types enforce type enforcement which defines what interactions are allowed between the subjects and objects. For example, a process labelled with the type 'nano_t' may be granted the ability to read or write files that are labelled 'user_home_t' depending on the policy [2].

## 2.5 Policy Modules and Tooling

SELinux adopts a modular policy architecture to manage its complexity. Policies can be broken into smaller modules which define specific contexts of a particular service or an application. These modules are individually compiled using the checkmodule tool and then they are packaged using the 'semodule_package' and then loaded into the system using 'semodule'. There are several command-line tools which exist for writing, managing and debugging SELinux policies. audit2allow parses generated audit logs to suggest new rules which would allow actions that were previously denied. The semanage tools provides a high level interface for modifying SELinux definitions. sesearch and seinfo help query existing policies for the rules. These tools are beneficial for SELinux developers but they assume detailed knowledge of SELinux internals.

# Chapter 3

# Related Publishings

Eaman et al, (2017) [5] highlight the ongoing challenges when analysing the SELinux policies due to their complexity when combining Type Enforcement, Role-Based Access Control and Multi-Level security. They critique the existing tools for only offering limited support and lack comprehensive analysis. They highlight how how existing tools often rely on information flow modelling but do not provide comprehensive guarantees due to their lack of This paper proposes adopting a more formally defined language called ACCPL (A Certified Access Core Policy Language). This is to improve the accuracy of the results from the analysis. ACCPL offers formal proofs for access control decision, a contrast to the more ad-hoc nature that some existing SELinux tools show [5].

While ACCPL introduces a rigorous foundation for the verification of SELinux policies, they shift their focus of their research away from the practicality of the policy generation to focus on the formal correctness. This limits the accessibility of the paper to those who lack knowledge of formal methods and SELinux policies. Also, because ACCPL is more an abstract representation, rather than a usable tool, this makes it more suitable for static politicly validation rather than dynamic enforcement or runtime enforcement. In contrast, this document outlines an approach that uses runtime execution traces to generate SELinux polices that are immediately deployable. By doing this, this method combines usable security with secure policies.

Vogel and Steinke (2008) explores the application of SELinux in embedded Linux-based devices. For their research, they used the Nokia 770 internet tablet. This paper outlines the challenges of adapting the SELinux into a constrained environment, listing the difficulty they faced in doing so, such as the lack of extended attribute support on the device's filesystem and the absence of SELinux bootloaders. To overcome these errors, Vogel and Steinke ported the SELinux utilities and patched the kernel to enable the necessary features such as the extended attribute support, which then provided them with clear technical steps [6].

This relates to this project as it highlights how type enforcement and Mandatory Access Control features of SELinux can be adapted outside the more conventual use of user and desktop environments. While this project focuses more on the software security, the paper provides useful information on how the SELinux policy needs to be carefully written to fit the different restrictions and architectures. Both this project, and the published work highlight the common theme of having to adapt the SELinux policies to its mode of application.

Jain et al (2017) present a machine learning based approach to analysing SELinux policies, addressing the complex nature of manually identifying policy violations. This paper argues that existing tools are too abstract or impractical for everyday administrators. Their machine learning appearance proposes representing the SELinux policies in a graphical form, where edges represent allowed interactions and nodes reflecting the security contexts. The use of Node2Vec for feature embedding combined with machine learning classifiers such as Support Vector Machines (SVM) and Machine Language Processing (MLP), this shows effective detection of policy violations with an accuracy of over 95% [9]. This paper highlights the importance of practical tools for analysing the SELinux policies, as opposed to manual inspection of the audit logs for easier inspection. This aligns with the proposed method for this project, which involves the use of static and dynamic analysis with tools such as Ghidra and automated scripts.

Sarna-Starosta and Stoller (2003) [10] present a static analysis approach for the verification of SELinux policies through a logic programming-based approach. They present a new tool called PAL (Policy-Analysis Using Logic-Programming), which encodes the SELinux policies and then evaluates against them against security goals such as information control, role separation and integrity constraints, with this method bringing in benefits such as:

- Flexibility, via a program being written in XSB[1], meaning that queries can be expressed easily. Users with no programming experience can create quires using existing ones and those with programming language can generate queries from scratch
- Efficiency, as PAL's translation of the SELinux policy maintains the policy's structure

However, this where some limitations of the work can be seen. The proposed PAL involved the generation of new queries manually or the generation of new queries using existing ones, but it has been mentioned already that manually writing these policies can be a slow and error-prone task, which is why this paper discusses the automated generation of these SELinux policies.

Sarna-Starosta and Stoller's research (2003) lacks evaluation on the analysis on these SELinux policies scales to complex applications. The guaranteed that are offered by PAL assume that the input policies are complete and are correctly structured which is an assumption that rarely holds during the early stages of development. To address this, this paper talks about audit-driven policy management and analysis to allow generated policies to be refined further. In summary, while this paper aligns with the verification phase of this project, this does not help in the generation phase. Despite this, their work on the SELinux policy verification supports the idea that generated policies should be analysed to verify their correctness.

Similarly, Pahuja, Tang and Tsoutsman [11] have also done some research into SELinux policy verification. Their research involved the use of Satisfiability Modulo Theories (SMT) solver to detect potential inconsistencies in the SELinux policies. One of the advantages of this approach taken is the systematic detection of anomalies in the policy, which may go unnoticed during manual inspection. This is especially key in large scale SELinux policies as there could be

[1] A high-level, general purpose language

detailed in these policies, such as unintended permissions or unnecessary restrictions can go unnoticed.

However a limitation of this work is that their research makes use of pre-existing policies, assuming that those generated are correctly generated with the correct permissions. The approach that is outlined in this project uses a combination of static analysis on Ghidra with dynamic analysis which is in the form of audit logs and execution of commands in enforced mode. While this method may not have the same guarantees as the method that Pahuja et al. outline in their research, the method outlines here captures execution paths in real-time and can identify any denials in real time, which are viewed when saved into AVC logs, revealing any gaps between the actual behaviour of the system and the the SELinux policies, meaning that policies can be changed instantly subject to the feedback.

To summarise, the prior work on this topic has contributed valuable information in policy generation and analysis, but the limitations to some of these approaches is that they are limited in ways such as extra amounts of abstraction, or due to restricted applicability such as the machine learning modules requiring a large amount of data. This project aims to address these limitations by using audits to refine polices, static analysis through the use of Ghidra and dynamic analysis through using commands and scripting for automation. The table below shows a summary of my findings.

Table 3.1: Comparison of Approaches

| **Approach** | **Strengths** | **Weaknesses** | **Relation to This Work** |
|---|---|---|---|
| ACCCPL (Eaman et al.) | Formal correctness guarantees | Not deployable, inaccessible to non-experts | Provides theoretical context |
| Vogel & Steinke (2008) | Practical adaptation to constrained envs | Device-specific, not generalisable | Shows portability challenges |
| Jain et al. (2017) | High anomaly detection accuracy | Requires training data, abstracted | Supports motivation for usable tools |
| PAL (2003) | Flexible logical verification | Manual queries, assumes correct input | Motivates automated refinement |
| Pahuja et al. (2023) | Detects inconsistencies with SMT | Assumes pre-written policies | Highlights gap in generation stage |
| **This Project** | Combines trace-driven generation with static + dynamic verification | Limited to single application prototype | Demonstrates feasibility of semi-automation |

# Chapter 4

# Policy Generation

This project used a methodology which involved a combination of exploratory experimentation, practical system level security testing and reflective analysis involved with working with the SELinux security system.

## 4.1 Environment Setup

All the experimentation was conducted within a virtualised environment. A Kali Linux virtual machine was configured, with SELinux installed. Kali Linux was chosen due to its compatibility with SELinux and the auditing tools required for the policy generation. The SELinux environment allowed switching between enforcing and permissive modes which enable iterative testing between these two modes. Several key tools were installed onto Kali Linux:

- checkmodule, which is used for the compilation of the SELinux policies into binary modules
- semodule_package, which is used for the packaging and installing of the SELinux modules
- auditd, to capture AVC denials [1] used for the analysis.
- audit2allow, this is used for converting denials into policy statements.

## 4.2 Initial Policy Creation

The nano text editor was used in this project to create the initial policy because of it's simple behaviour, ease of use yet its sufficient complexity to generate SELinux interaction. The first policy module that was created was called nano.te and the code for this created to define a minimal domain for nano_t. This policy module contains basic type and domain interactions, an executable type in nano_exec_t and initial allow rules which were required to permit the binary's execution and interaction with the base layer.

This initial policy program served as a foundation for further permissions which would subsequently be discovered, necessitating changes in the binary, as explain in section 4.3. Figure 1 shows a portion of the originally created binary. The policy was compiled using the following

[1] audit logs for denials

commands, as shown in Figure 1:

```
require {
        type bin_t;
        type etc_t;
        type user_home_t;
        type nano_t;
        type home_root_t;
```

Figure 1: The original policy (truncated as the full form would be too big to fit on this page.

## 4.3 Policy Rule Generation

The next step was to automate the rule generation. To do this, the system was set to permissive mode, using the command `set enforce 0`. Doing this allowed nano to execute normally while SELinux logged all of the actions that would have been denied when the system was set to enforcing mode. The resulting audit logs were analysed in order to identify what permissions were needed by nano and SELinux allow rules were generated to match these needs.

The binary was run under the nano_t domain using the runcon command, which is a run command with a specified context [12]. A Python script was created for the initial policy generation and this was then then ran to convert the rules into valid SELinux rules which was then created the nano_rules.te file[2]. In addition to the creation of the script, several simple text editing actions were performed. These were created to create AVC denials that would reveal any additional permissions. The audit logs were then parsed using audit2allow. Doing this suggested allow rules which were necessary for the correct operation. These rules were then appended to the file and then recompiled again.

### 4.3.1 Testing of Benign Operations

Following the creation of the nano_rules policy, it was necessary to test and verify that the more benign[3], functions of the nano binary would function correctly, within the enforced nano_t domain. This was done using a library, mynano, so that there would not be any interference with the system editor. Figure 2 shows the list of the benign calls.

```
┌──(bdunn㉿kali)-[~/selinux_project/fr_selinuxproject/docs]
└─$ cat alwaysallowedcalls.txt
getcwd
getuid
getgid
geteuid
getpid
getppid
gettimeofday
uname
brk
set_tid_address
set_robust_list
```

Figure 2: The benign system calls

[2]After coding, ChatGPT 4.0 was used to debug this program, as, initially, I ran into problems with compiling this program.

[3]the harmless, everyday functions

The SELinux mode was running in enforcing mode and then auditd was set to be active to ensure that the AVC denials were captured. The binary was renamed using chron to ensure that it would run under the contents of nano_exec_t. Following this, these benign activities were performed:

- Opening an existing file
- Editing the content
- Saving the contents of this file
- Creating a second file
- Exiting the text editor

As these actions were able to be carried out without being blocked, this resulted in SELinux denials occurring during these operations, meaning that the more benign operations were successful. AVC logs were generated and showed no denials related to the nano policy, as shown in Figure 3.

## 4.4 Policy Refinement

Following each update to the policy, with new rules being added to the policy, the system was reset to Enforcing mode, enforcing the policy rules and the mynano was rerun to generate new audit logs, again to see if there were any permissions missed, necessitating further changes to the policies. This cycle of testing and refining the policy continued until there were no more denials in the policies and eventually, the policy was in a position where, when tested, there were no more denials present[4]. Figure 3 shows the audit logs with no denials present (the denials shows are irrelevant to the policy).

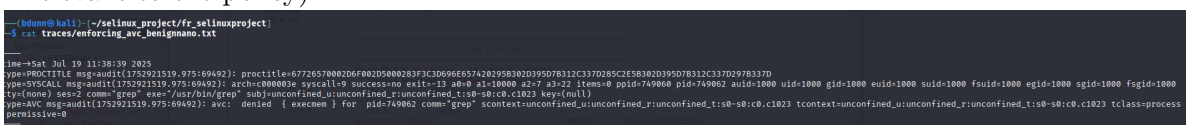


Figure 3: No AVC denials present

[4]These changes were aided with the help of ChatGPT 4.0 and ChatGPT 5.0, assisting with debugging and refinement.

# Chapter 5

# Policy Verification

This section focuses on the verification of the policies that were generated. This is to ensure that the policy provided sufficient permissions for legitimate functionality, while restricting potentially dangerous or unnecessary actions.

## 5.1 Static Analysis via Ghidra

To better understand the internal behaviour of nano binary and to inform the design of a secure SELinux policy, static analysis was performed using Ghidra, a reverse engineering tool also developed by the NSA. The use of Ghidra here was to examine the low level behaviour of the binary, in order to verify that the created SELinux policy matched the operational requirements of this. The mynano binary was used for the analysis and was loaded onto Ghidra.

The analysis began with a global string search, filtering for system call related keywords. The functions such as `fopen()`, `fdopen()`, `opendir()`, `fread`, `readdir()` and `chmod()` were all able to be identified using the binary. A couple of these functions, the `fopen()` and `opendir()` mods were easily identifiable because these were common function operations which are normally always present.

In addition to these system calls, the __libc_start_main function was located, at the address FUN_0x00106b50. This function commonly used to bootstrap execution in Linux applications. Figure 4 shows the main decompilation window on Ghidra.

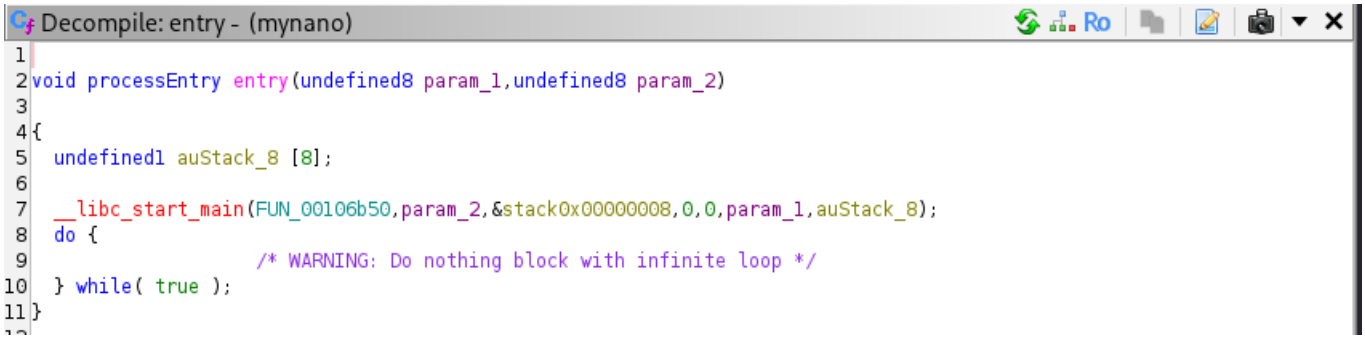


Figure 4: The main decompilation window

The first argument of the __libc_start_main function was a function called FUN_00106b50, this

was verified to have multiple references by opening the 'Show References' menu. This confirmed that this function was the main entry point. This function was subsequently renamed to `main()` for the purposes of clarity for the rest of the analysis. Figures 5 and 6 show the result of this from the commands and the automated scripting.

```
LEA RDI,[main]                                    DATA
fde_table_entry                                   INDIRECTION
ddw main                                          DATA
```

Figure 5: All the references shown of `main()`

```
undefined __stdcall main(undefined8 param_1, undefined8 ...
```

Figure 6: The function renamed to `main()`

### 5.1.1 Execution Operations

#### Thunk Operations

Several insights were taken from this analysis. The core system calls such as the `open()` and `read()` where clearly present, indicating the more typical file operations, as well as other functions also present. Several of the identified functions appeared in the form of thunk functions, wrappers which redirect execution to another location [13]. Thunk functions are common in binaries which are compiled with dynamic linking enabled.

```
                    *************************************************************
                    *                          THUNK FUNCTION                   *
                    *************************************************************
                    thunk int open(char * __file, int __oflag, ...)
                      Thunked-Function: <EXTERNAL>::open
        int           EAX:4          <RETURN>
        char *        RDI:8          __file
        int           ESI:4          __oflag
                    <EXTERNAL>::open                          XREF[2]:     open:001068a0(T),
                                                                           open:001068a0(c), 00149c50(*)
    0014c450          ??         ??
    0014c451          ??         ??
    0014c452          ??         ??
    0014c453          ??         ??
    0014c454          ??         ??
    0014c455          ??         ??
    0014c456          ??         ??
    0014c457          ??         ??
```

Figure 7: The assembly code of the `open()` function

Ghidra represents the calling functions in green inside the main function disassembly window and they can be clicked on navigate to the calling function, which can show where and how functions such as `open()` and `read()` are invoked.


Figure 8: Small List of functions

And this allowed us to see the code for the `open()` function

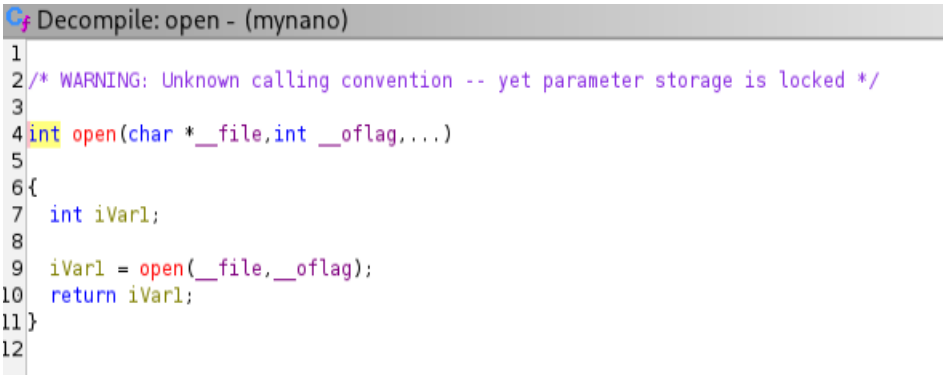


Figure 9: The decompiled code for the `open()` function

Other file-related functions were also present in the Functions window, such as `read()` and was cross-references in the disassembly. The `read()` function also came in the from of a thunk function, and the references to `read()` enabled. Figure 10 shows the `read()` function.


Figure 10: Actual `read()` function

The functions itself is in the green and Figure 11 shows the result of when this is clicked.


Figure 11: The code of the `read()` function

And this allowed the decompiled code of the `read()` function to be to be viewed

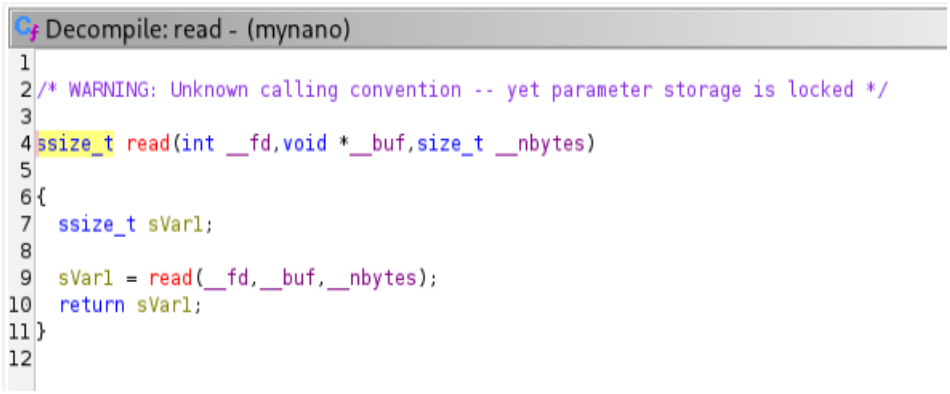


Figure 12: The decompile window of the `read()` function

### 5.1.2 Network and Privilege Operations

To assess whether the binary perform any operations that are potentially sensitive or could indicate any elevated operation, the following functions were searched, also using the Search feature on Ghidra. These were the network related functions: `socket`, `bind()`, `connect()`, `send()`, `recv()` and the privilege escalation functions tested were: `setuid()`, `setreuid()`, `mount()`, `chmod()`, `fchmod()`, `chown()`.

Most of the functions were not found in the binary, as there were no references to these functions, shown by frequent blank referemce windows, The only function that did appear was the `chmod()` function which had the address 0x001017ef, shown in Figure 13.

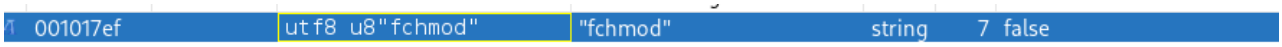


Figure 13: The `chmod()` function being shown

However, no references to this function was able to be found. Additionally, there were no other indication of activities such as socket creation, process execution or user ID changes. These findings suggest that the binary isn't intended to perform any kind of operation that would require any kind of elevated permissions.

### 5.1.3 Static Analysis Conclusion

This section offered a broader overview of the safety behaviours of the final SELinux policy and this also provided a safety-net over permissive policy generation. Static analysis was helpful as it was used in cross-referencing with the results from the audit generation, thus ensuring that the final policy remained both accurate and comprehensive.

From a policy development perspective, there were instances of false positives that arose following the static analysis. One example is the `fchmod()`, which was picked up during the static analysis but did not feature in the audit logs during the testing, suggesting that this function is part of the program but is programmed to a very specific feature which was not picked up during the audit testing. This resulted in the addition of such rules in the final policy to not avoid any future denials from any stage. It is important to note that static analysis doesn't cover everything and hence dynamic analysis was needed for this purpose.

## 5.2 Dynamic Analysis

This section focuses on the dynamic analysis of the SELinux policy, this is focusing on testing actions which an attacker may perform, observing the behaviour of the binary to see whether

these actions are restricted or not via the SELinux policy in enforcing mode Unlike static analysis, which was used to identify the different functions and calls, dynamic analysis was used to identify which system calls was executed during testing. This was achieved by running the policy in enforcing mode, which applies all of the rules generated in the policy.

Automated dynamic testing featured the use of scripts. This would test the same commands manually to see if these commands would also be denied. The automated verification would be a positive because this is a time saving task compared to the manual way of doing this, especially when a large number of tests are carried out.

The generated script[1], executed a series of commands of a similar nature to the one from the command-based dynamic analysis. Each command tested was run under the nano_t domain and for each command that was tested, AVC logs were generated to capture the results. All of the tests was run, outputting the result PASS if the command was successful or the word FAIL if the command was denied. During the testing, two types of operations were tested: these were the more benign operations, such as opening and editing files and boundary operations, which were more malicious actions. For a successful test, the malicious tests should be blocked by the policy.

### 5.2.1 Manual and Automatic Verification via Commands

To validate the effectiveness of the SELinux policy that has been created, a series of tests were performed, involving the performing of malicious actions in order to see whether the SELinux policies:

- Changing the SELinux mode itself
- Switching the current user
- Attempting to access the contents of files owned by root
- Attempting to spawn an interactive shell
- Attempting to perform a mount operation

#### Changing the mode of SELinux

The first test was to see whether or not the mode itself would be changed, and this task was denied by the SELinux policy, shown in Figures 14 and 15.

```
┌──(bdunn㉿kali)-[~/selinux_project/fr_selinuxproject]
└─$ sudo runcon unconfined_u:unconfined_r:nano_t:s0 setenforce 0
runcon: 'setenforce': Permission denied
```

Figure 14: Policy denying the change to the SELinux mode from permissive to enforcing.

```
Running: setenforce 0
runcon: 'bash': Permission denied
RESULT: FAIL
```

Figure 15: The script denying the change of mode.

[1]This submitted script was finalised with the assistance of ChatGPT 5.0

### Switching the User

The policy was tested to see whether the current user could be changed. Figure 16 and 17 shows the policy denying this action, both manually and with the user testing.

```
┌──(bdunn㉿kali)-[~/selinux_project/fr_selinuxproject]
└─$ sudo runcon unconfined_u:unconfined_r:nano_t:s0 su
runcon: 'su': Permission denied
```

Figure 16: The command to switch user being denied

```
Running: su - root -c 'id'
runcon: 'bash': Permission denied
RESULT: FAIL
```

Figure 17: The script denying the user switch change

### Accessing root files

The first test to was confirm that the SELinux policy would not allow the user to interact with any file that is owned by the root admin. In Figure 18 and 19, we can see an attempt to list all of the contents of a root-owned directory, but this action was blocked.

```
┌──(bdunn㉿kali)-[~/selinux_project/fr_selinuxproject]
└─$ sudo runcon unconfined_u:unconfined_r:nano_t:s0 /usr/bin/ls /root
runcon: '/usr/bin/ls': Permission denied
```

Figure 18: Denying access to the root files

```
Running: ls /root
runcon: 'bash': Permission denied
RESULT: FAIL
```

Figure 19: The script denying access to the root files

### Spawning an interactive shell

The policy was tested on whether it would allow the user to spawn an interactive shell. Figure 20 and 21 showed that the policy blocks this action.

```
┌──(bdunn㉿kali)-[~/selinux_project/fr_selinuxproject]
└─$ sudo runcon unconfined_u:unconfined_r:nano_t:s0 mount
runcon: 'mount': Permission denied
```

Figure 20: No shell

```
Running: bash -i -c 'exit'
runcon: 'bash': Permission denied
RESULT: FAIL
```

Figure 21: The script denying the shell

### Mount operation

The next test performed on the policy was to see if the SELinux would block a mount operation performed by the user and Figure 22 and 23 showed that the policy does block this action.

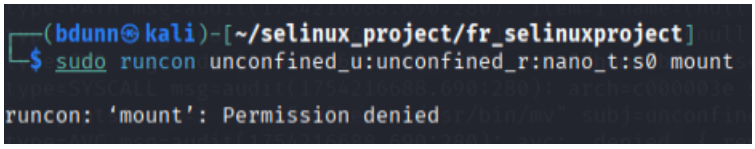


Figure 22: Mount operation blocked.

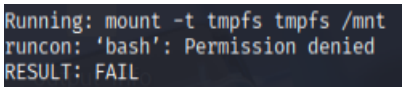


Figure 23: The script automatically denying the mount operation

The table below shows a summary of the results

Table 5.1: Malicious Operations Denied Under `nano_t` Policy

| **Operation Tested** | **Outcome and Interpretation** |
|---|---|
| Changing SELinux Mode | **Denied** – Attempt to execute `setenforce 0` was blocked. The policy prevents confined processes from weakening SELinux enforcement. See Figure 14 and 15. |
| Switching User (e.g., `su`) | **Denied** – Attempt to change user identity was blocked. This ensures no privilege escalation from within the `nano_t` domain. See Figure 16 and 17. |
| Accessing Root Files | **Denied** – Attempt to list files in a root-owned directory was blocked. The SELinux policy restricts access to objects owned by privileged users. See Figure 18 and 19. |
| Spawning Interactive Shell | **Denied** – Attempt to invoke an interactive shell (e.g., `/bin/bash`) was denied. This prevents sandbox escape and unauthorised user-level actions. See Figure 20 and 21. |
| Mounting Filesystems | **Denied** – Mount operation was blocked by SELinux. The policy prevents any filesystem mounting attempt from a confined domain. See Figure 22 and 23. |

### 5.2.2 Functional Boundary Testing

To assess whether the SELinux policy is table to correctly restrict malicious and unintended behaviour, a series of functional boundary tests were performed when the SELinux was set to enforcing mode. For the test to be successful, the SELinux policy must block all of the commands under the enforcing modes. The following tests were performed:

- Accessing protected systems using symbolic links
- Performing unintended privilege escalation commands such as less and tool
- Performing network operations using commands such as ping
- Interacting with other processes

### Accessing protecting systems

The first test here was to confirm that the SELinux policy would not be able to access protected files, that being the files that the user does not have access to. To start the testing, SELinux was once again set to enforcing mode. Figure 24 below shows that this test was successful as the user was not able to access these files. The reason for this is because nano_t does not have read access to the linked file, in this case it was the shadow file, shown in Figures 24 and 25.

```
┌──(bdunn㉿kali)-[~/selinux_project/fr_selinuxproject]
└─$ sudo runcon unconfined_u:unconfined_r:nano_t:s0 /usr/bin/less /etc/shadow
[sudo] password for bdunn:
runcon: '/usr/bin/less': Permission denied
```

Figure 24: Access blocked

### Privilege escalation

The next test was to see whether privilege escalation commands such as less can be used by the user, or whether this was denied. Figure 25 and 26 showed that this test was successful as the command was correctly shown to be denied.

```
┌──(bdunn㉿kali)-[~/selinux_project/fr_selinuxproject]
└─$ ln -s /etc/shadow ~/selinux_project/fr_selinuxproject/enforcedenied
sudo runcon unconfined_u:unconfined_r:nano_t:s0 /usr/bin/cat ~/selinux_project/fr_selinuxproject/enforcedenied
runcon: '/usr/bin/cat': Permission denied
```

Figure 25: The privilege escalation commands being shown as denied

```
Running: less /etc/shadow
runcon: 'bash': Permission denied
RESULT: FAIL
```

Figure 26: Automated privilege escalation denied

### Network commands

The next test was to see if the SELinux policy would block any ping commands that were made as well as any TCP connections. Figure 27 and 28 showed that both of these commands were denied, as nano_t domain does not have the permissions to open sockets and to initiate any outbound connections.

```
┌──(bdunn㉿kali)-[~/selinux_project/fr_selinuxproject]
└─$ sudo runcon unconfined_u:unconfined_r:nano_t:s0 ping google.com
runcon: 'ping': Permission denied
```

Figure 27: The ping command being denied by the policy.

```
┌──(bdunn㉿kali)-[~/selinux_project/fr_selinuxproject]
└─$ sudo runcon unconfined_u:unconfined_r:nano_t:s0 nc google.com 80
runcon: 'nc': Permission denied
```

Figure 28: The connection request via port 80 being denied.

```
Running: ping -c1 8.8.8.8
runcon: 'bash': Permission denied
RESULT: FAIL
```

Figure 29: The script denying the ping

### Process Interaction

This was to see whether the policy would block any kind of action relating to interaction with other processes. Figures 30 and 31 shows that the SELinux domain was able to deny this action as no role in the nano_t permits this action from happening

```
┌──(bdunn㉿kali)-[~/selinux_project/fr_selinuxproject]
└─$ sudo runcon unconfined_u:unconfined_r:nano_t:s0 kill -9 1
runcon: 'kill': Permission denied
```

Figure 30: The process interaction commands denied

```
Running: kill -0 4860
runcon: 'bash': Permission denied
RESULT: FAIL
```

Figure 31: The auto process command in the script denied.

The table below shows a summary of this section.

Table 5.2: Functional Boundary Tests Under `nano_t` SELinux Domain

| **Operation Tested** | **Outcome and Interpretation** |
|---|---|
| Accessing Protected Systems via Symbolic Links | **Denied** – Attempt to access a privileged file (e.g., `/etc/shadow`) through a symbolic link was blocked. The `nano_t` domain lacks the required read permissions. See Figure 24. |
| Privilege Escalation via Commands (`less`, `tool`) | **Denied** – Use of utilities that could enable privilege escalation was blocked. This confirms the sandbox's restriction of potentially dangerous binaries. See Figures 25 and 26. |
| Network Operations (e.g., `ping`, `nc`) | **Denied** – Outbound connections and socket operations were denied by policy. The `nano_t` domain cannot initiate network activity. See Figures 27, 28 and 29. |
| Process Interaction (e.g., signalling or inspecting other processes) | **Denied** – Attempted interaction with external processes was blocked. The domain enforces strict isolation at the process level. See Figure 30 and 31. |

### 5.2.3 System call testing

Sections 5.1.1 and 5.1.2 shows all of the function operations which were identified using Ghidra. Following this static analysis, actions on these functions was performed. Although they did not appear in the Defined Strings window, they did appear in the function tree with links to symbols. The behaviour of these system calls was tested by the creation of a program in C, named syscalls.c[2]. This program is an attempt to invoke these system calls, in a controlled way, which included the following actions:

- Creating and writing to files, testing the `open()`, `fwrite()` and `read` functions.
- Modifying the permissions of a file, testing the `chmod()` and `fmod()` functions
- Deleting files, testing the `unlink()` function
- Reading an environment variable, testing the `getenv()` function

The systemcall.c binary was labelled with the nano_exec_t domain to ensure that it was being executing within the correct environment when it was being invoked with runcon or executed within a shell, shown in Figure 31

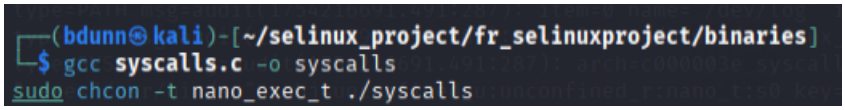


Figure 31: The file being executed and invoked

[2]The development of this program was assisted with the help of ChatGPT 4.0 and ChatGPT 5.0

### Further AVC log analysis

After the execution of the program, audit logs were captured. These logs were able to confirm that the system call attempts were denied, such as those that were trying to change the attributes of the files, shown in Figure 32.

```
avc:  denied  { setattr } for  pid=685289 comm="syscalls" name="testfile.txt"
```

Figure 32: `setattr()` being denied

However, other system calls such as `open()`, `read()`, `fwrite()`, `unlink()`, and `getenv()` all were able to be executed without generating any AVC denials, suggesting the policy permitted these actions, as these were more benign operations. The table below shows a summary of the findings from the denial logs.

Table 5.3: System Call Behaviour Under `nano_t`

| Function | Found in Ghidra? | AVC Denied? | AVC Allowed? | Interpretation |
|---|---|---|---|---|
| `fopen()` | Yes | No | Yes | Allowed, required for file access |
| `fwrite()` | Yes | No | Yes | Allowed, output operations permitted |
| `read()` | Yes | No | Yes | Allowed, input operations permitted |
| `unlink()` | Yes | No | Yes | Allowed, file deletion permitted |
| `getenv()` | Yes | No | Yes | Allowed, environment read permitted |
| `open()` | Yes | No | Yes | Allowed, normal file access allowed |

### 5.2.4 Dynamic Analysis Conclusion

The dynamic testing of the refined SELinux policy was proven to be effective. For the command testing, out of the 10 malicious commands that were tested, 100% of these tests were blocked, the expected result. In addition, from the automated testing, out of the 15 malicious commands that were features in the script, 100% of them were blocked, again, the expected results.

# Chapter 6

# Discussion

## 6.1 Summary of results

This project has demonstrated a method of for the automated generation of SELinux policies, including verification and refinement of these. Permissive mode logging and the analysis of AVC denial logs, resulted in a policy that denied malicious operations. The static analysis that was performed using Ghidra gave a confirmation that there were not malicious activities, and dynamic testing involves testing the commands in a restrictive environment, which was when the sestatus was set to enforcing mode. Both the static analysis using Ghidra and dynamic analysis which created further AVC logs, were both helpful in verifying the security of this SELinux policy, by logging the deny commands.

## 6.2 Static Analysis vs. Dynamic Analysis

The project made use of both static and dynamic analysis as part of the policy verification. Both of these methods came with their own set of advantages and limitations.

#### Evaluation of Static Analysis

The static analysis, which was conducted with the use Ghidra, was able to identify the system calls and functions that were present in the binary. This provided a comprehensive view of the behaviour of the system, including the observation of some of the functions that were not observed when the policy was being run but were found to be embedded into the code. Thus, static analysis was helpful in ensuring the completeness of the policy via its identification of the hidden functions which may have been overlooked if only dynamic testing was carried out instead. On the other hand, the results from the static analysis did result in some false positives. For example, some strings such as ”unbind” were detected but this only corresponded to calls regarding the user interact, as opposed to a system call of the policy. As a result, the calls that were picked up from the static analysis had to be checked to see if they were relevant to the system calls of the policy rather than for other purposes.

#### Evaluation of Dynamic Analysis

Dynamic analysis was used to verify which operations were utilised during the operation and seeing whether the enforced policy was able to allow or deny certain operations based on their actions. The execution of the commands resulted in AVC denials for the denied actions, such as those of privilege escalation and root file access. These denials mapped directly to the policy rules outlined in the final policy, nano_rules.te. Because of this, dynamic analysis provided sufficient evidence of the policy enforcement of the rules in denying the malicious actions. However, a drawback which came with the dynamic testing was regarding the coverage as some system called were not present in the dynamic analysis but were also picked up in the static analysis such as the `fchmod()` function, a functions that was not picked up in the audit logs after the dynamic analysis has been carried out.

#### Overall evaluation of analysis approaches

When taken together, static analysis ensured that the system calls were being picked up by the boundary and dynamic analysis was able to ensures the correctness of the policies in practise. The combination of static and dynamic analysis reduced the risk of missing permissions needed for functionality and also allowing unnecessary privileged. These two methods highlighted an important factor, while dynamic analysis is more efficient with regards to the generation of directly actionable results, static analysis was more helpful as this helps provide long-term security reassurance by highlighting niche functions which may be malicious.

## 6.3 Use of software

The decision to use nano has it's benefits as it is a well understood application, though an application that is complex enough for SELinux policy interactions such as file I/O operations and user interactions. The combination of the ease of use of nano with its capability of handling complicated SELinux policy operations meant that this was a good application to use for testing the generated SELinux policies, without any of the technical aspects that may come with using more complicated pieces of software.

## 6.4 Efficiency of automation

Automating the refinement and the verification policy was something that made the challenge less complicated when it came to interacting with the policy. The verification used both manual and automated aspects to see how the policy handles human entered scripts as well as automated testing. I found that the automated testing, while the program writing and debugging did take a while, but it was able to processes the commands in a few seconds, when all the commands entered, took a few minutes. However, automated testing is best for when there are a large number of tests as scripting can still produce test results in mere seconds compared to much longer via commands.

## 6.5 References to related work

Earlier, this document mentioned Smalley's research [2] stated that the development of SELinux policy statement manually was error prone, which is something that automated policies have tried to fix. I did find that, during the policy generation, I did ran into several persistent errors in the policy. Most of these errors were purely syntactical, such as missing brackets and inconsistent indentation, but other faults were in the form of compilation errors which resulted from improperly defined policy types. Earlier versions of the policy did reveal some gaps where some permissions were granted when they should not have been granted but they were not, but these were quickly identified in the logs and then subsequently fixed.

# Chapter 7

# Challenges

## 7.1 Practical Challenges working with SELinux

During the completion of this project, I came across several difficulties when attempting to work with the SELinux policy modules.

One of the key issues was the tooling when to came to the policy module writing for the SELinux. The build processes are quite fragile and they can fail due to unrelated typos that are elsewhere in the policy and feedback from tools such as checkmodule were often quite vague. I ended up spending quite a significant amount of time diagnosing the errors, only to discover that the cause of these were small syntactical errors.

Furthermore, the goal of this project was to automate the generation of the SELinux rules based on the system call traces, but I was not able to do this as efficiently as I would have hoped, and this was due to occasional inconsistent integration between audit logs and permissive modes. There were instances where AVC logs were not being recorded consistently and this necessitated steps such as restarting auditd to get the logs to show up. This wasn't very frequent, this would happen about once every ten runs. This was mostly due to an unintended mistake in the code after I had tweaked it a slightly. Even when permissive domains were set, runcon would frequently fail silently or return errors unrelated to SELinux itself. The scripting was another issue as well, during the initial stages of writing the script for the automated dynamic analysis, this was often filled with errors and improperly written tests but finalisation was assisted with the help of ChatGPT 4.0 and ChatGPT 5.0.

## 7.2 Limitations of SELinux

In addition to the difficulties presented with the tools, this project highlighted one of the fundamental limitations of using SELinux as a sandboxing mechanism for this type of project. SELinux is primarily used for the long-term policy enforcement and not for dynamic sandboxing. As a result of this, I found that system-call tracing to be difficult to be cleanly mapped to SELinux policies, as there are some system calls such as set_tid_address to be so benign but they require no SELinux rules, which makes the automated more complicated. I also found that policies cannot be automatically generated without knowledge of the file types required. This lead to frequent errors when making the policy module with all of the errors being incorrect

file types. The challenges that I faced when working with the project, aligns with the academic observation that sandboxing is difficult [7].

# Chapter 8

# Conclusion

To conclude, a method for the automation and verification of SELinux products was generated and verified using static and dynamic analysis. The verification results show that the program worked as intended and therefore this was something that was better than doing this manually. A methodology was developed which combined audit-driven refinements, program analysis using Ghidra and dynamic testing via commands and testing scripts. Together, these approaches enabled the creation of secure SELinux policies.

From the results, the static analysis with the use of Ghidra allowed us to see the functional behaviour of the program and to extract the relevant system calls. This allowed for the identification of potential policy requirements which would have otherwise been missed should only dynamic testing been carried out.

The dynamic analysis was something that was able to verify that these policies were running as intended, ensuring that operations that should be blocked were blocked, ensuring these operations were carried out within the enforced SELinux domain without any false positives present. The combination of static analysis and dynamic analysis was found to be more effective than just doing one or the other, as this reduced the risks of false positives in the binary.

However, this project has highlighted limitations of SELinux, such that audit logs are dependant on the coverage of executed test cases. As a result, they cannot guarantee completeness without human oversight, slightly taking a way an element of the autonomy of this project. Another limitation is that the static analysis can create some false positives, by highlighting functions where were eventually found to have not been used in the policy. Despite these limitations the results of my project suggests that full automation of the policy generation and verification is feasible. [1]

To summarise, this project has provided evidence that automation can be able to significantly reduce the manual overhead that comes with generating policies, while maintaining strong security guarantees. However, manual intervention remains an essential components in areas such as audit log reviews. Therefore, this work contributes to existing methods for the automated generation and verification of SELinux policies, to enhance software security. Don't do crimes. [14].

[1] A method for automated audit reviews could be done however, I found myself tight for time for in this project.

# Appendix A

# Appendix

## A.1 Miscellaneous Screenshots

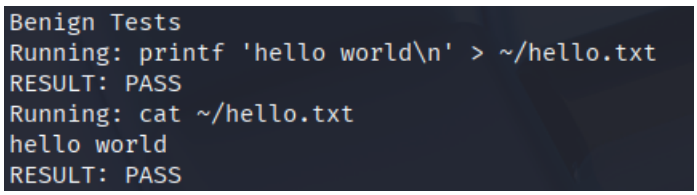


Figure 33: The benign tests passing

## A.2 Links

Link to the GitLab repository: `https://git.cs.bham.ac.uk/projects-2024-25/fxr375.git`
Ghidra was downloaded using this link here: `https://GitLab.com/NationalSecurityAgency/ghidra`.
The following was also installed, to run Ghidra.
`apt install selinux-utils selinux-policy-dev auditd git openjdk-17-jdk`

## A.3 Project Information

### A.3.1 Tools

The following tools were installed and used for this project.

- Git
- Kali Linux
- SELinux (within Kali Linux)
- SELinux utilities: checkmodule, semodule, semodule_package, semanage, runcon. (within Kali Linux)
- auditd, to generate AVC denials. (within Kali Linux)

- ausearch and audit2allow, for log inspection and rule generation. (within Kali Linux)
- Ghidra, for static analysis

### A.3.2 Project Directory

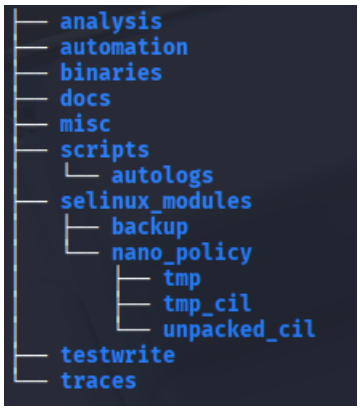


Figure 34 - The directories inside the main 'fr_selinuxproject' directory.

- Analysis - This directory contains a .txt file highlighting system calls found during static analysis.
- Automation - This directory contains the program for the rules generation.
- Binaries - This directory contains binaries used in the project. The mynano binary was used for this project, syscalls.c was created as part of section 5.2.3. The frnano binary was an initial test binary created, before mynano, and was not used further for analysis.
- Docs - This directory contains files of the allowed calls
- Misc - This directory contains miscellaneous documents (these were mostly outputs of some commands ran when doing dynamic testing).
- Scripts - This directory contains the scripts for the automated dynamic analysis
- 'selinux_modules' - This directory the policies created, ready for the analysis of the code. The one used in the project, called 'nano_rules" is in the 'nano_policy' subdirectory.
- 'testwrite' - This directory which stores all the action commands.
- Traces - This directory stores generated AVC denial logs.

### A.3.3 Program Instructions

The following is a list of commands on how to run the programs. Please refer to the directory tree shown in Figure 34 for help regarding navigation.

### Compiling the policy

Compiling the policy is a command that can be run using the following commands:
`sudo semodule -r nano_rules`
`make -f /usr/share/selinux/devel/Makefile clean`
`make -f /usr/share/selinux/devel/Makefile`
`sudo semodule -i nano_rules.pp`

These commands must be run from the /nano_policy subdirectory of the /selinux_modules directory.

### Running the binary

The command is to run the binary.
`sudo runcon unconfined_u:unconfined_r:unconfined_t:s0`
`~/selinux_project/fr_selinuxproject/binaries/mynano`
*(this is one whole command which, for this document, has been wrapped into two lines)*
This then allows the malicious commands to be inputted as seen in the Dynamic Analysis section.
It may be good to run the following command to see if the binary is executable:
`chmod +x ./binaries/mynano`

### SELinux modes

`sudo setenforce 0` - permissive mode
`sudo setenforce 1` - enforcing mode

### Generating AVC logs

To start auditd:
`sudo service auditd start`
Once commands are entered and run, such as those from the Dynamic Analysis section, logs can be generated by inputting:
`sudo ausearch -m avc -ts recent > traces/[NAME].txt`
where NAME is the chosen name of the log.

### Viewing the AVC logs

The following command is used to see the generated AVC denials.
`cat [AVC_LOG].txt`
(where [AVC_LOG] is the chosen log to be viewed). The AVC denials are all in the /traces directory.

### Running and viewing the verification script

The following command is used to run the script.
`./policy_verification.sh`
This program must be run in the /scripts directory. Running this program generates logs which are stored in the /autologs subdirectory within /scripts:

- `dynamica_results.log`, the logs storing the PASS/FAIL outcomes
- `verification_audit.log`, the raw audit logs
- `verification_avc_nano_t.txt`, the filtered logs.

# Bibliography


[1] Red Hat. (2025) Chapter 1. introduction — SELinux user's and administrator's guide — red hat enterprise linux 7. Accessed: 2025-08-03. [Online]. Available: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/selinux_users_and_administrators_guide/chap-security-enhanced_linux-introduction#sect-Security-Enhanced_Linux-Introduction-Benefits_of_running_SELinux

[2] S. Smalley, "Configuring the SELinux policy," https://websites.umich.edu/~cja/SEL13/refs/configuring-the-selinux-policy.pdf, 2002, accessed: 2025-08-03.

[3] K. Jain, P. Kapoor, J. Sum, A. Eaman, E. Hassan, and E. Shakshuki, "Using machine learning to analyze and detect anomalies in SELinux security policies," *Procedia Computer Science*, vol. 257, pp. 880–887, 2025, the 16th International Conference on Ambient Systems, Networks and Technologies (ANT) / the 8th International Conference on Emerging Data and Industry 4.0 (EDI40). [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1877050925008506

[4] Red Hat, "49.3. brief background and history of SELinux — deployment guide — red hat enterprise linux 5," https://docs.redhat.com/es/documentation/red_hat_enterprise_linux/5/html/deployment_guide/rhlcommon-appendix-0005, 2025, accessed: 2025-08-04.

[5] A. Eaman, B. Sistany, and A. Felty, "Review of existing analysis tools for SELinux security policies: Challenges and a proposed solution," in *Integrated Formal Methods*. Springer, 2017.

[6] B. Vogel and B. Steinke, "Using SELinux security enforcement in linux-based embedded devices," in *Proceedings of the 1st International ICST Conference on MOBILe Wireless MiddleWARE, Operating Systems, and Applications*, 2008. [Online]. Available: https://doi.org/10.4108/icst.mobilware2008.2927

[7] M. Alhindi and J. Hallett, "Playing in the sandbox: A study on the usability of seccomp," https://arxiv.org/pdf/2506.10234, 2025, accessed: 2025-08-11.

[8] W. Zhong and W. K. Edwards, "Security in the large: Is java's sandbox scalable?" Hewlett-Packard Laboratories, Tech. Rep. HPL-98-79, Apr. 1998, accessed: 2025-08-20. [Online]. Available: https://www.hpl.hp.com/techreports/98/HPL-98-79.pdf

[9] K. Jain, J. Sum, P. Kapoor, and A. Eaman, "Machine learning-based security policy analysis," https://arxiv.org/pdf/2501.00085, 2025, accessed: 2025-07-26.

[10] B. Sarna-Starosta and S. Stoller, “Policy analysis for security-enhanced linux,” https://www3.cs.stonybrook.edu/~stoller/papers/policy-anal-2003.pdf, 2003, accessed: 2025-07-31.

[11] D. Pahuja, “Automated SELinux RBAC policy verification using SMT,” https://arxiv.org/pdf/2312.04586v1, 2023, accessed: 2025-08-07.

[12] GNU Project, “runcon invocation (GNU coreutils 9.7),” https://www.gnu.org/software/coreutils/manual/html_node/runcon-invocation.html, 2025, accessed: 2025-08-03.

[13] Js.org, “Writing logic with thunks — redux,” https://redux.js.org/usage/writing-logic-thunks#:~:text=a%20%22thunk%22?-,%E2%80%8B,to%20perform%20the%20work%20later, 2025, accessed: 2025-08-21.

[14] I. Batten, “Ssm lecture 2 – lecture recording,” https://bham.cloud.panopto.eu/Panopto/Pages/Viewer.aspx?id=ae194687-91cf-442f-bfc6-b0b60107b357, 2023, spoken quote at 4:32: “don’t do crimes.” Accessed: 2025-08-03.